AI Assisted Coding Lab 16.4

Veekshitha Adharasani

2303A52175

Batch – 41

Question 1: Hospital Management Database

Prompt:

Create a complete SQL solution for a Hospital Management System by designing a well-structured database with three tables: Doctors, Patients, and Appointments. Define primary keys and foreign keys, and use constraints like NOT NULL and UNIQUE with data types that are compatible with SQLite. Then, write SQL queries using joins to display all appointments for a specific doctor using doctor_id, retrieve the full history of a patient using patient_id, and count the total number of different patients treated by each doctor. Also, include sample INSERT statements to test the database and a few SELECT queries to verify that the data is correct. Ensure that the final SQL code is clean, well-organized, and executes in SQLite without any errors.

Code:

```sql
-- Create the Doctors table
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    specialty TEXT NOT NULL,
    UNIQUE(first_name, last_name)
);

-- Create the Patients table
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth DATE NOT NULL,
    UNIQUE(first_name, last_name, date_of_birth)
);
-- Create the Appointments table
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
-- Sample INSERT statements to test the database
INSERT INTO Doctors (first_name, last_name, specialty) VALUES ('John', 'Doe', 'Cardiology');
INSERT INTO Doctors (first_name, last_name, specialty) VALUES ('Jane', 'Smith', 'Neurology');
INSERT INTO Patients (first_name, last_name, date_of_birth) VALUES ('Alice', 'Johnson', '1980-05-15');
INSERT INTO Patients (first_name, last_name, date_of_birth) VALUES ('Bob', 'Brown', '1990-08-22');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES (1, 1, '2024-07-01');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES (1, 2, '2024-07-02');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES (2, 1, '2024-07-03');
-- SQL query to display all appointments for a specific doctor using doctor_id
SELECT a.appointment_id, d.first_name AS doctor_first_name, d.last_name AS doctor_last_name,
       p.first_name AS patient_first_name, p.last_name AS patient_last_name, a.appointment_date
```

```sql
-- Create the Appointments table
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
-- Sample INSERT statements to test the database
INSERT INTO Doctors (first_name, last_name, specialty) VALUES ('John', 'Doe', 'Cardiology');
INSERT INTO Doctors (first_name, last_name, specialty) VALUES ('Jane', 'Smith', 'Neurology');
INSERT INTO Patients (first_name, last_name, date_of_birth) VALUES ('Alice', 'Johnson', '1980-05-15');
INSERT INTO Patients (first_name, last_name, date_of_birth) VALUES ('Bob', 'Brown', '1990-08-22');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES (1, 1, '2024-07-01');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES (1, 2, '2024-07-02');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES (2, 1, '2024-07-03');
-- SQL query to display all appointments for a specific doctor using doctor_id
SELECT a.appointment_id, d.first_name AS doctor_first_name, d.last_name AS doctor_last_name,
       p.first_name AS patient_first_name, p.last_name AS patient_last_name, a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;
-- SQL query to retrieve the full history of a patient using patient_id
SELECT a.appointment_id, d.first_name AS doctor_first_name, d.last_name AS doctor_last_name,
       a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = 1;
-- SQL query to count the total number of different patients treated by each doctor
SELECT d.first_name AS doctor_first_name, d.last_name AS doctor_last_name, COUNT(DISTINCT a.patient_id) AS total_patients
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
```

Output:

SQL < 1 / 1 > 1 - 2 of 2

appointment_id	first_name	last_name	first_name	last_name	appointment_date
1	John	Doe	Alice	Johnson	2024-07-01
2	John	Doe	Bob	Brown	2024-07-02

SQL < 1 / 1 > 1 - 2 of 2

appointment_id	first_name	last_name	appointment_date
1	John	Doe	2024-07-01
3	Jane	Smith	2024-07-03

SQL < 1 / 1 > 1 - 2 of 2

doctor_id	first_name	last_name	COUNT(DISTINCT a.patient_id)
1	John	Doe	2
2	Jane	Smith	1

Explanation:

The database is created using three connected tables, where the Appointments table connects the Doctors and Patients tables using foreign keys. This helps keep the data organized and avoids duplication. SQL joins are used to combine information from these tables so that we can easily get details like doctor appointments, a patient's history, and the total number of patients treated by each doctor.

Question 2: Real-Time Applica on: Online Shopping Database

Prompt:

Create a complete SQL solution for an online shopping (E-commerce) system by designing a well organized database with four tables: Users, Products, Orders, and OrderDetails. Define primary keys and foreign keys, use suitable data types, and apply constraints like NOT NULL and UNIQUE so that it works properly in SQLite. Make sure the tables are properly connected and the database follows good design rules (up to Third Normal Form). Then, write SQL queries using joins to get all orders made by a specific user using user_id, find the product that was purchased the most based on quantity, and calculate the total revenue for a particular month. Also include sample INSERT statements to test the database and some SELECT queries to check the data. Finally, add a few simple suggestions to improve database design and make queries run faster. Ensure the entire SQL code is clear, well-structured, and runs without errors.

Code:

```sql
-- Create Users table
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL
);

-- Create Products table
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
    stock INTEGER NOT NULL
);
-- Create Orders table
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    order_date TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
-- Create OrderDetails table
CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
-- Sample INSERT statements
INSERT INTO Users (username, email, password) VALUES ('john_doe', 'john@example.com', 'password123');
INSERT INTO Products (product_name, price, stock) VALUES ('Laptop', 1000.00, 10);
INSERT INTO Orders (user_id, order_date) VALUES (1, '2023-10-01');
```

```sql
CREATE TABLE OrderDetails (
    quantity INTEGER NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
-- Sample INSERT statements
INSERT INTO Users (username, email, password) VALUES ('john_doe', 'john@example.com', 'password123');
INSERT INTO Products (product_name, price, stock) VALUES ('Laptop', 1000.00, 10);
INSERT INTO Orders (user_id, order_date) VALUES (1, '2023-10-01');
INSERT INTO OrderDetails (order_id, product_id, quantity) VALUES (1, 1, 2);
-- Query to get all orders made by a specific user using user_id
SELECT o.order_id, o.order_date, p.product_name, od.quantity
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE o.user_id = 1;
-- Query to find the product that was purchased the most based on quantity
SELECT p.product_name, SUM(od.quantity) AS total_quantity
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_id
ORDER BY total_quantity DESC
LIMIT 1;
-- Query to calculate the total revenue for a particular month (e.g., October 2023)
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE strftime('%Y-%m', o.order_date) = '2023-10';
-- Suggestions to improve database design and query performance:
-- 1. Add indexes on foreign key columns (user_id in Orders and product_id in OrderDetails) to speed up join operations.
-- 2. Consider adding a status column to the Orders table to track order states (e.g., pending, shipped, delivered) for better order management.
-- 3. Implement a more secure password storage mechanism (e.g., hashing) instead of storing plain text passwords in the Users table.
```

Output:

SQL ▾ < 1 / 1 > 1 - 2 of 2

```
SELECT o.order_id, o.order_date, od.product_id, od.quantity, od.price
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE o.user_id = 1;
```

order_id	order_date	product_id	quantity	price
1	2026-03-18 13:18:04	1	1	999.99
1	2026-03-18 13:18:04	2	2	29.99

SQL ▾ < 1 / 1 > 1 - 1 of 1

```
SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id
ORDER BY total_quantity_sold DESC
LIMIT 1;
```

product_name	total_quantity_sold
Mouse	2

SQL ▾ < 1 / 1 > 1 - 1 of 1

```
SELECT SUM(od.quantity * od.price) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE strftime('%Y-%m', o.order_date) = '2024-01';
```

total_revenue
NULL

Explanation:

The database is designed using multiple connected tables, where the OrderDetails table links the Orders and Products tables. This helps keep the data well-organized and avoids repeating the same information. SQL joins and aggregate functions are used to combine and analyze the data, making it easy to track user orders, find the most popular products, and calculate the total revenue for each month efficiently.

Question 3: Library Database

Prompt:

Create a complete SQL solution for a Library Management System by designing a well-organized database with three tables: Books, Members, and Loans. Define primary keys and foreign keys, use suitable data types, and apply

constraints like NOT NULL and UNIQUE so that it works correctly in SQLite. Make sure the tables are properly connected and the database follows good design rules (up to Third Normal Form). Then, write SQL queries using joins to get all books that are currently issued and not yet returned, find overdue books where the loan date is more than 30 days from today, and count how many books each member has borrowed. Also, suggest a simple indexing strategy to improve performance. Include sample INSERT statements to test the database, and ensure the final SQL code is clear, well-structured, and runs without any errors.

Code:

```sql
-- Create the Books table
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER,
    ISBN TEXT UNIQUE NOT NULL
);
-- Create the Members table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    Email TEXT UNIQUE NOT NULL,
    JoinDate DATE NOT NULL
);
-- Create the Loans table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    MemberID INTEGER NOT NULL,
    BookID INTEGER NOT NULL,
    LoanDate DATE NOT NULL,
    ReturnDate DATE,
    FOREIGN KEY (MemberID) REFERENCES Members (MemberID),
    FOREIGN KEY (BookID) REFERENCES Books (BookID)
);
-- Sample INSERT statements to test the database
INSERT INTO Books (Title, Author, PublishedYear, ISBN) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 1925, '978-0743273565'),
('To Kill a Mockingbird', 'Harper Lee', 1960, '978-0061120084'),
('1984', 'George Orwell', 1949, '978-0451524935');
INSERT INTO Members (FirstName, LastName, Email, JoinDate) VALUES
('John', 'Doe', 'john.doe@example.com', '2023-01-01'),
('Jane', 'Smith', 'jane.smith@example.com', '2023-01-01');
INSERT INTO Loans (MemberID, BookID, LoanDate) VALUES
(1, 1, '2023-01-10'),
```

```sql
    FOREIGN KEY (BookID) REFERENCES Books (BookID)
);
-- Sample INSERT statements to test the database
INSERT INTO Books (Title, Author, PublishedYear, ISBN) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 1925, '978-0743273565'),
('To Kill a Mockingbird', 'Harper Lee', 1960, '978-0061120084'),
('1984', 'George Orwell', 1949, '978-0451524935');
INSERT INTO Members (FirstName, LastName, Email, JoinDate) VALUES
('John', 'Doe', 'john.doe@example.com', '2023-01-01'),
('Jane', 'Smith', 'jane.smith@example.com', '2023-01-01');
INSERT INTO Loans (MemberID, BookID, LoanDate) VALUES
(1, 1, '2023-01-10'),
(1, 2, '2023-01-15'),
(2, 3, '2023-01-20');
-- Query to get all books that are currently issued and not yet returned
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
-- Query to find overdue books where the loan date is more than 30 days from today
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND l.LoanDate < DATE('now', '-30 days');
-- Query to count how many books each member has borrowed
SELECT m.FirstName, m.LastName, COUNT(l.LoanID) AS BooksBorrowed
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID, m.FirstName, m.LastName;
-- Indexing strategy to improve performance
CREATE INDEX idx_loans_memberid ON Loans (MemberID);
CREATE INDEX idx_loans_bookid ON Loans (BookID);
CREATE INDEX idx_loans_loan_date ON Loans (LoanDate);
```

Output:

SQLite

SQL ▾ 1 / 1 1 - 2 of 2

Title	Author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

SQL ▾ 1 / 1 1 - 2 of 2

Title	Author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

SQL ▾ 1 / 1 1 - 2 of 2

FirstName	LastName	BooksLoaned
John	Doe	1
Jane	Smith	1

Explanation:

The system is designed using three connected tables, where the Loans table links the Books and Members tables. This helps keep the data organized and avoids repeating the same information. SQL joins and conditions are used to combine and analyze the data, making it easy to track which books are issued, find overdue books, and calculate how many books each member has borrowed.

Question 4: Optimize Queries

Prompt:

Analyze and improve the performance of SQL queries using an example. Start by writing the original query that uses a subquery to find books written by authors from the UK, and then rewrite it using a more efficient method with joins. Make sure both queries give the same result. Briefly explain how the improved query is faster and works better for large data. Also suggest simple improvements like adding indexes on important columns (such as author_id and country) and avoiding unnecessary subqueries. Provide both the original and improved queries, along with a short step to check that the results are correct. Ensure the final output is clear, simple, and works properly in SQLite.

Code:

```
Original Query using Subquery:sql
SELECT title
FROM books
WHERE author_id IN (
    SELECT author_id
    FROM authors
    WHERE country = 'UK'
);
Improved Query using Joins:
sl
SELECT b.title
FROM books b
JOIN authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
### Explanation of Performance Improvements
The improved query using joins is generally faster than the original query with a subquery because it allows the database engine to optimize the execution plan more effec
To further enhance performance, you can create indexes on the `author_id` column in the `books` table and the `country` column in the `authors` table. This will allow the
CREATE INDEX idx_author_id ON books(author_id);
CREATE INDEX idx_country ON authors(country);
```
```
### Checking ResultsTo ensure that both queries return the same results, you can run both queries and compare their outputs.```sql
-- Run the original query   SELECT title
FROM books  WHERE author_id IN (
    SELECT author_id
    FROM authors
    WHERE country = 'UK'
);
-- Run the improved query   SELECT b.title
FROM books b JOIN authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```You can use a simple method to check if the results are the same, such as counting the number of rows returned by each query or using a `UNION` to find any discrepanci
-- Check if both queries return the same results   SELECT title
FROM books
WHERE author_id IN (
    SELECT author_id
```
```

```
SELECT b.title
FROM books b
JOIN authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
### Explanation of Performance Improvements
The improved query using joins is generally faster than the original query with a subquery because it allows the database engine to optimize the execution plan more effec
To further enhance performance, you can create indexes on the `author_id` column in the `books` table and the `country` column in the `authors` table. This will allow the
CREATE INDEX idx_author_id ON books(author_id);
CREATE INDEX idx_country ON authors(country);

### Checking ResultsTo ensure that both queries return the same results, you can run both queries and compare their outputs.```sql
-- Run the original query   SELECT title
FROM books  WHERE author_id IN (
    SELECT author_id
    FROM authors
    WHERE country = 'UK'
);
-- Run the improved query   SELECT b.title
FROM books b JOIN authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```You can use a simple method to check if the results are the same, such as counting the number of rows returned by each query or using a `UNION` to find any discrepanci
-- Check if both queries return the same results   SELECT title
FROM books
WHERE author_id IN (
    SELECT author_id
    FROM authors
    WHERE country = 'UK'
)UNION
SELECT b.title
FROM books b JOIN authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```If the result of the `UNION` query returns no rows, it means both queries are returning the same set of titles, confirming that the improved query is correct. This meth
```

Output:

Explanation:

The original query uses a subquery, which may be slower when working with large amounts of data, while the optimized query uses a join to make it run faster. Adding indexes and avoiding nested queries further improves performance, while still giving the same results.

Question 5: University Course Registration

Prompt:

Create a complete SQL solution for a University Course Registration System by designing a well-organized database with four tables: Students, Courses, Faculty, and Registrations. Define primary keys and foreign keys, use suitable data types, and apply constraints like NOT NULL and UNIQUE so that it works correctly in SQLite. Set up relationships so that each student can register for multiple courses and each course is taught by one faculty member, while following good design rules (up to Third Normal Form). Then, write SQL queries using joins to list all students enrolled in a specific course using course_id, find faculty members who teach more than two courses, and get the courses with the highest number of registrations. Also include sample INSERT statements for testing, and ensure the final SQL code is clear, well-structured, and runs without any errors.

Code:

```sql
-- Create the Students table
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);

-- Create the Faculty table
CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Create the Courses table
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    faculty_id INTEGER NOT NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);
-- Create the Registrations table
CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)
);

-- Sample INSERT statements for testing
INSERT INTO Faculty (first_name, last_name, email) VALUES ('John', 'Doe', 'john.doe@university.edu');
INSERT INTO Students (first_name, last_name, email) VALUES ('Alice', 'Smith', 'alice.smith@university.edu');
INSERT INTO Courses (course_name, faculty_id) VALUES ('Introduction to Computer Science', 1);
```

```sql
CREATE TABLE Courses (
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);
-- Create the Registrations table
CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)
);

-- Sample INSERT statements for testing
INSERT INTO Faculty (first_name, last_name, email) VALUES ('John', 'Doe', 'john.doe@university.edu');
INSERT INTO Students (first_name, last_name, email) VALUES ('Alice', 'Smith', 'alice.smith@university.edu');
INSERT INTO Courses (course_name, faculty_id) VALUES ('Introduction to Computer Science', 1);
INSERT INTO Registrations (student_id, course_id) VALUES (1, 1);
-- Query to list all students enrolled in a specific course using course_id
SELECT s.first_name, s.last_name, s.email
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
WHERE r.course_id = 1;
-- Query to find faculty members who teach more than two courses
SELECT f.first_name, f.last_name, COUNT(c.course_id) AS course_count
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING COUNT(c.course_id) > 2;
-- Query to get the courses with the highest number of registrations
SELECT c.course_name, COUNT(r.registration_id) AS registration_count
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY registration_count DESC
LIMIT 1;
```

Output:

SQL ▼ < 1 / 1 > 1 - 1 of 1

first_name	last_name	email
Alice	Smith	alice.smith@university.edu

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 1 of 1

course_name	registration_count
Introduction to Computer Science	1

Explanation:

The Registrations table connects the Students and Courses tables, allowing a student to enroll in multiple courses and handling the many-to-many relationship in an efficient way. This design helps keep the database well-organized and avoids duplicate data. SQL joins and aggregation functions are used to analyze enrollments, understand faculty workload, and identify popular courses. Proper use of primary and foreign keys ensures data integrity, while constraints like NOT NULL and UNIQUE improve data accuracy. Additionally, indexing important columns can improve query performance, making the system faster and more efficient when working with large amounts of data.